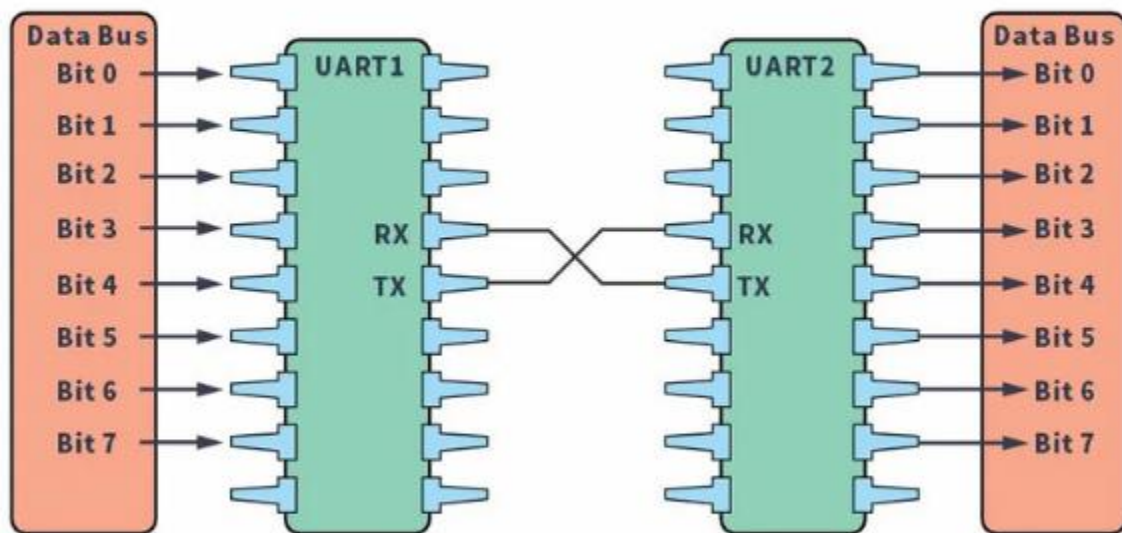


UART_Project

UART Communication



Prepared by:
Haneen Fady Kamel Shahin

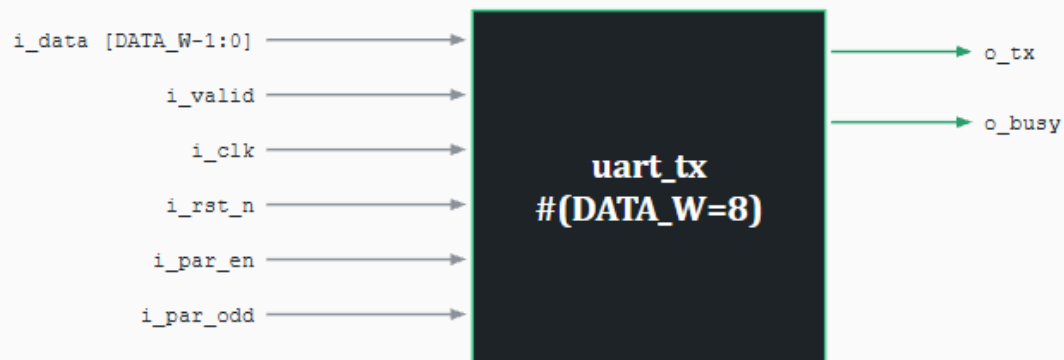
TX:

Overview:

1. RTL DATA FLOW & TOP-LEVEL ARCHITECTURE

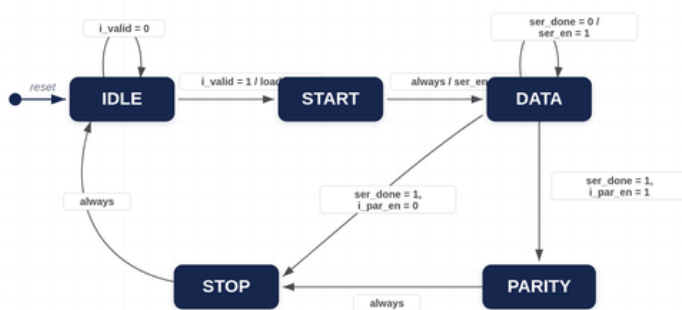
The UART Transmitter (uart_tx) is a parameterizable digital IP designed to convert parallel input data (i_data) into an asynchronous serial bitstream (o_tx).

uart_tx — Port Diagram



The hardware datapath and control flow proceed as follows:

TX_FSM — State Transition Diagram



condition / output • self-loops indicate hold behavior • "always" = unconditional transition

A. IDLE State & Parallel Load:

- When the system is IDLE and a valid data flag (`i_valid = 1`) is asserted, the TX_FSM asserts the internal load signal.
- The Serializer module samples `i_data` into its internal shift register (`shift_reg <= i_data`) and resets its internal bit counter to zero (`counter <= 0`).
- The `paritybit_calc` module pre-calculates and registers the parity bit on the assertion of load.

B. Transmission Framing & State Sequence:

- **START State:** The FSM drives `mux_sel = 2'b00`, routing `start_bit (1'b0)` to `o_tx` via the output multiplexer.
- **DATA State:** The FSM asserts `ser_en = 1` and `mux_sel = 2'b01`.
 - The Serializer outputs `shift_reg[0]` (LSB first) onto `serial_bit`.
 - On each clock cycle, `shift_reg` shifts right by 1 bit, and counter increments until all `DATA_W` bits are shifted out.
 - Upon shifting the last bit, Serializer asserts the `ser_done` pulse.
- **PARITY State (Optional):** If parity is enabled (`i_par_en = 1`), the FSM transitions to PARITY (`mux_sel = 2'b10`).
 - The pre-calculated `parity_bit` is driven to `o_tx`.
 - The `paritybit_calc` module outputs even parity (XOR) if `i_par_odd = 0` or odd parity (XNOR) if `i_par_odd = 1` over `i_data`.
- **STOP State:** The FSM drives `mux_sel = 2'b11`, routing `stop_bit (1'b1)` to `o_tx`.
 - **Handshaking:** If `i_valid` is asserted high during the STOP state, load is asserted immediately to pre-load the next data payload for back-to-back framing.

2. SUBMODULE INTERCONNECTIVITY:

- TX_FSM: Controls state progression and generates control signals (`load`, `ser_en`, `mux_sel`, `o_busy`) based on inputs `i_valid`, `i_par_en`, and `ser_done`.
- Serializer: Parameterized payload shifter generating `serial_bit` and `ser_done`.
- `paritybit_calc`: Combinational/registered parity generator outputting `parity_bit`.
- mux: Output routing multiplexer driving final `o_tx`.

3. VERIFICATION & TEST CASE STRATEGY:

To fully verify the `uart_tx` module, testbenches typically evaluate the following scenarios:

A. Reset Verification:

- Assert reset (`i_rst_n = 0`) to ensure FSM returns to IDLE, `o_busy` is deasserted (0), and shift counters are zeroed.

B. Parity Mode Verification (Even & Odd):

- Even Parity (`i_par_en = 1`, `i_par_odd = 0`): Verify parity bit equals XOR of `i_data`.
- Odd Parity (`i_par_en = 1`, `i_par_odd = 1`): Verify parity bit equals XNOR of `i_data`.

C. Standard Frame Transmissions (Parity Disabled):

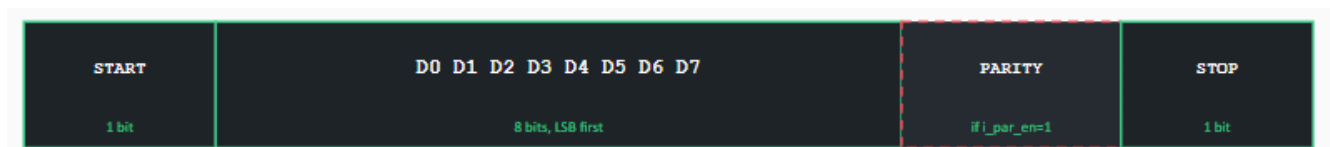
- Set `i_par_en = 0`, apply a valid payload (`i_data`) with `i_valid = 1`.
- Verify start bit (0), N data bits (LSB to MSB), and stop bit (1) timing.

D. Back-To-Back Transmissions:

- Assert `i_valid` while the current frame is in the STOP state.
- Verify that load asserts immediately, bypassing IDLE to transmit the next frame without line delay.

E. IDLE Line Stability:

- Ensure `o_tx` holds high (1'b1) continuously while in IDLE and `o_busy` remains 0.



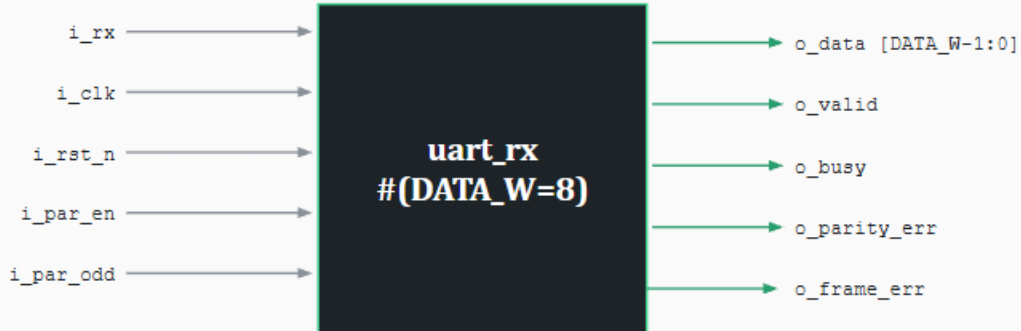
RX:

Overview:

1. RTL DATA FLOW & TOP-LEVEL ARCHITECTURE

The UART Receiver (`uart_rx`) is a parameterizable digital IP designed to convert an incoming asynchronous serial bitstream (`i_rx`) into a parallel output payload (`o_data`) along with status flags for frame validation.

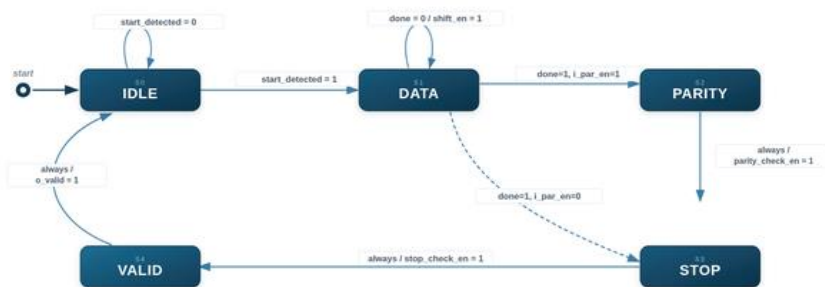
uart_rx — Port Diagram



The hardware datapath and control flow proceed as follows:

RX_FSM — State Transition Diagram

UART RECEIVER FINITE STATE MACHINE



A. IDLE State & Start Bit Detection:

- When the system is **IDLE**, the receiver monitors the incoming serial line (`i_rx`).
- Upon detecting a falling edge (`i_rx = 0`), the **RX_FSM** transitions to the **START** state and enables sampling/oversampling logic to validate the start bit center.

- If the start bit is verified low at the center sample point, frame reception commences. Otherwise, the state machine aborts and returns to `IDLE` (glitch filtering).

B. Reception Framing & State Sequence:

- **START State:**

The FSM asserts start-bit sampling logic. Once validated, the FSM transitions to the `DATA` state and enables the deserializer/shift register.

- **DATA State:**

The FSM asserts `deser_en = 1`.

- The `Deserializer` samples incoming serial bits at mid-bit intervals and shifts them into an internal register (LSB first).
- On each bit sample, internal bit counting increments until all `DATA_W` bits are gathered.
- Upon capturing the last bit, the `Deserializer` asserts the `deser_done` pulse.

- **PARITY State (Optional):**

If parity is enabled (`i_par_en = 1`), the FSM transitions to `PARITY`.

- The incoming bit on `i_rx` is sampled and compared against the calculated parity of the received payload via `paritybit_chk`.
- If a mismatch occurs, the parity error flag (`o_parity_err`) is asserted.

- **STOP State:**

The FSM transitions to `STOP` to sample the stop bit (`1'b1`).

- **Framing Check:** If `i_rx` is sampled low during the `STOP` state, a framing error flag (`o_stp_err/o_frame_err`) is asserted.
- **Output Latching:** Upon successful frame reception, the parallel output flag (`o_valid`) pulses high for one clock cycle, updating `o_data`.

2. SUBMODULE INTERCONNECTIVITY:

- `RX_FSM`: Controls receiver state progression and generates enable signals based on line activity, frame parameters, and module completion flags.
- `Deserializer`: Parameterized serial-to-parallel converter that samples `i_rx` and shifts bits into the parallel payload register.

-
- `paritybit_chk`: Calculates expected parity across received data bits and checks for discrepancies against the sampled parity bit.
 - `Data_Sync / Sampler`: Edge detection and oversampling synchronization unit filtering noise and determining accurate bit sampling points.

3. VERIFICATION & TEST CASE STRATEGY:

To fully verify the `uart_rx` module, testbenches typically evaluate the following scenarios:

A. Reset & Glitch Verification:

- Assert reset (`i_rst_n = 0`) to ensure FSM returns to `IDLE`, `o_valid` is low, and flags/counters are cleared.
- Inject short negative pulses (glitches) on `i_rx` in `IDLE` to verify glitch rejection without triggering false reception.

B. Parity Mode Verification (Even & Odd):

- Even Parity (`i_par_en = 1, i_par_odd = 0`): Verify correct reception when parity matches, and assertion of `o_parity_err` when a parity bit error is injected.
- Odd Parity (`i_par_en = 1, i_par_odd = 1`): Verify correct odd-parity checking and error detection.

C. Standard Frame Reception (Parity Disabled):

- Drive a valid UART frame on `i_rx` with `i_par_en = 0`. Verify `o_data` matches transmitted payload and `o_valid` pulses high.

D. Framing Error Detection:

- Drive a frame where the stop bit is corrupted (`i_rx = 0` during `STOP` state). Verify `o_stp_err` is asserted.

E. Back-To-Back Frame Reception:

- Drive sequential frames with zero inter-frame gap. Verify that the receiver continuously latches consecutive payloads without losing sync.

4. FUNCTIONAL SIMULATION & VERIFICATION RESULTS:

To validate the physical functionality and protocol compliance of the design, RTL behavioral simulation was performed using **Siemens QuestaSim**.

A. Testbench Execution & Waveform Analysis:

- **Signal Alignment:** As observed in the QuestaSim waveform window, all internal state transitions (cs/ns), handshake pulses (load, ser_en, ser_done), and output framing (o_tx / o_rx) strictly adhere to the expected timing specification.
- **Data Integrity:** Serialized bit sequences accurately match the input parallel payload (i_data / o_data), verifying correct LSB-first transmission/reception and parity bit calculation.

B. Simulation Transcript Summary:

- **Test Case Coverage:** The simulation transcript confirms successful execution across all programmed verification scenarios, including standard data frames, parity check modes (even/odd), and back-to-back transfer timing.
- **Pass/Fail Verdict:** The testbench self-checking logic completed with **0 Errors** and **0 Warnings**.

